

# Image and Video Generations

Lecture 6 Supplement: Magic3D

劉育綸

Yu-Lun (Alex) Liu



# Recap: Score Distillation Sampling (SDS)

- Drop the noise predictor Jacobian to save computation time and memory.
- The final SDS gradient is thus defined as follows:

$$\nabla_{\theta} \mathcal{L}_{\text{SDS}}(\theta) = (\hat{\epsilon}_{\theta}(\mathbf{x}_t, y, t) - \epsilon_t) \frac{\partial \mathbf{x}_t}{\partial \theta}$$

# Recap: Connections Between DDIM and SDS

## DDIM Reserve Process

## SDS

Until convergence, repeat:

For each time step  $t = T, \dots, 1$ , repeat: 

1. Compute  $\mathbf{x}_{0|t} = \frac{1}{\sqrt{\bar{\alpha}_t}} \left( \mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \hat{\mathbf{e}}_{\theta}(\mathbf{x}_t, t) \right)$

2. Compute  $\tilde{\mu} = \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_{0|t} + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \hat{\mathbf{e}}_{\theta}(\mathbf{x}_t, t)$ .

3. Sample  $\mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ .

4. Compute  $\mathbf{x}_{t-1} = \tilde{\mu} + \sigma_t \mathbf{z}_t$ .

SDS:

Maximum  
Variance

1.  $t \sim \mathcal{U}(1, T)$ .  $c \sim \mathcal{U}(\mathcal{C})$ .

2.  $\mathbf{x}_{0|t} = g(\theta; c)$ .

3.  $\mathbf{z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ .

4.  $\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \mathbf{x}_{0|t} + \sqrt{1 - \bar{\alpha}_{t-1}} \mathbf{z}_t$ .

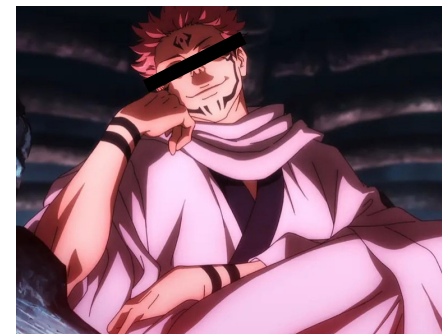
5.  $\theta \leftarrow \theta - \frac{\partial}{\partial \theta} \|\hat{\mathbf{e}}_{\theta}(\mathbf{x}_{t-1}, t) - \mathbf{z}_t\|^2$ .

DDIM 是 update  $\mathbf{x}_{0|t}$ , SDS 是  
update 一個 proxy representation  
(例如 NeRF) 的參數  $\theta$

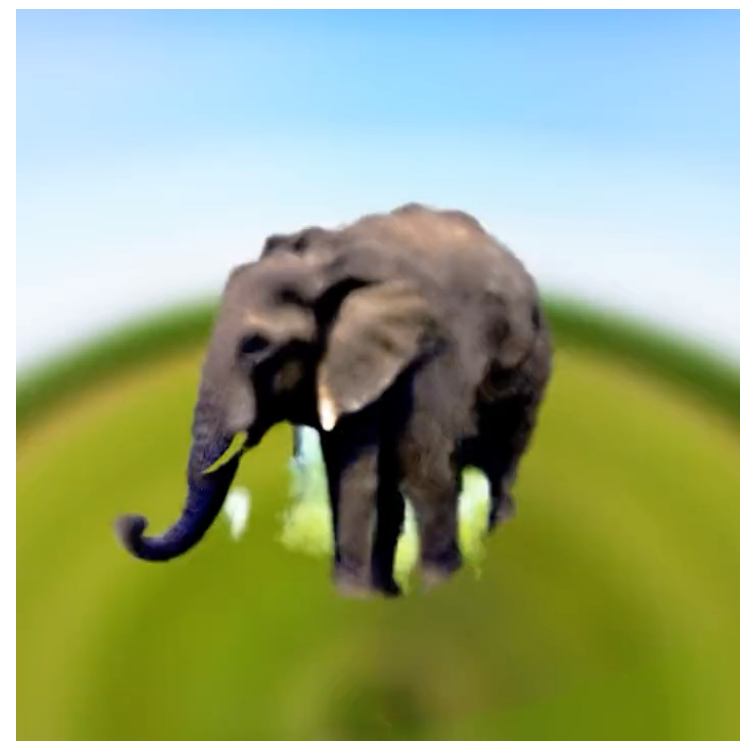
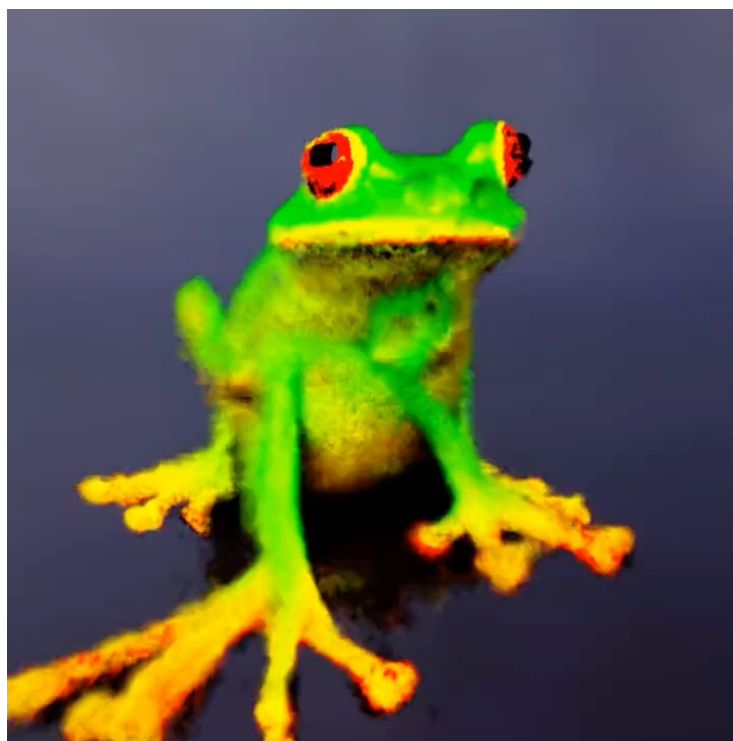
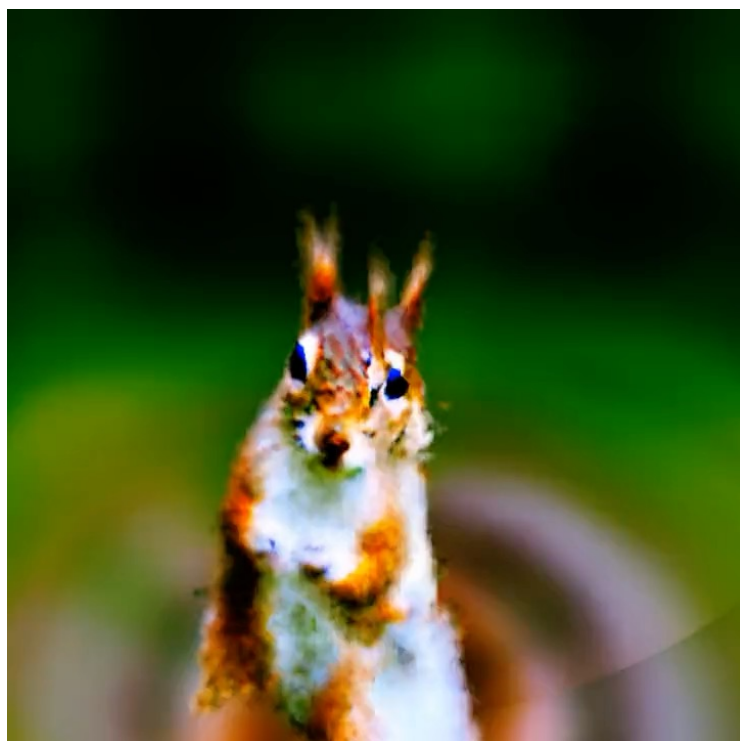
# Recap: Limitation of SDS

## Janus Problem

→ 雅努斯  
羅馬神話中的兩面之神



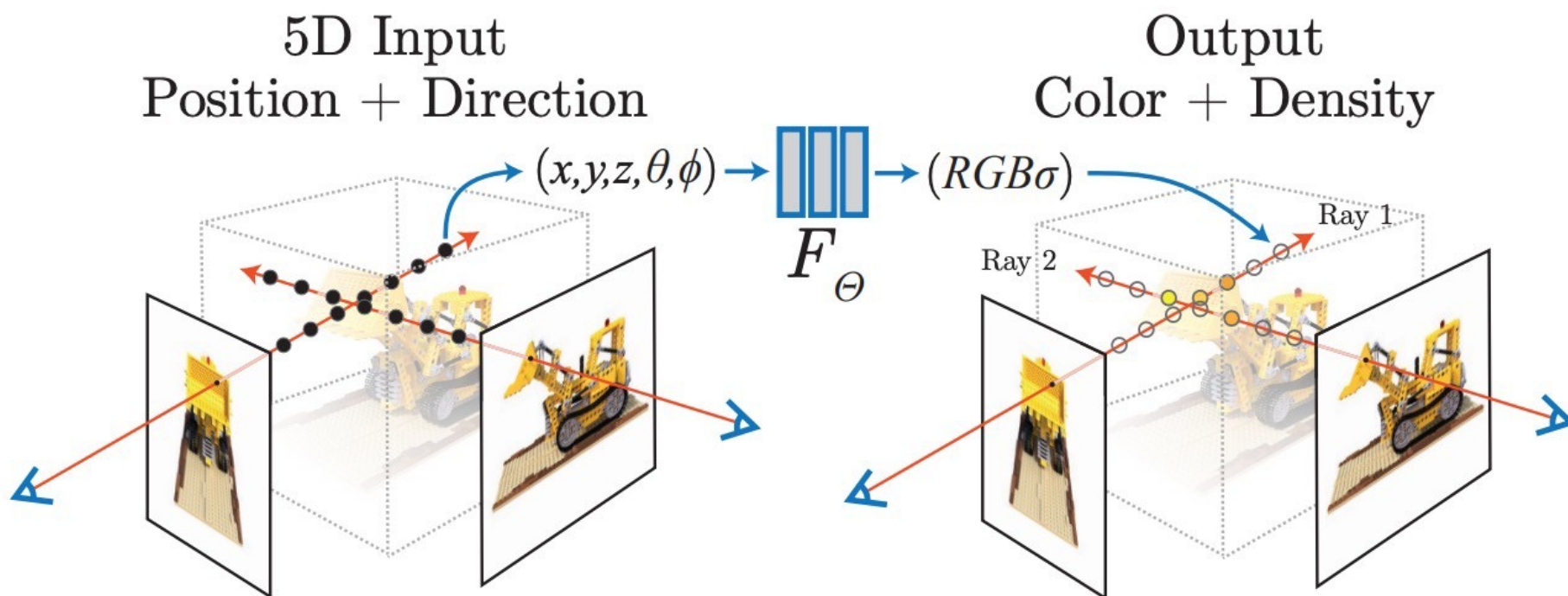
不是他



# Recap: 3D Representations: NeRF

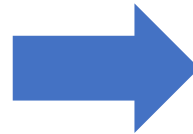
## Neural Radiance Fields (NeRF):

**Implicit** representation



# Recap: 3D Representations: Gaussian Splats

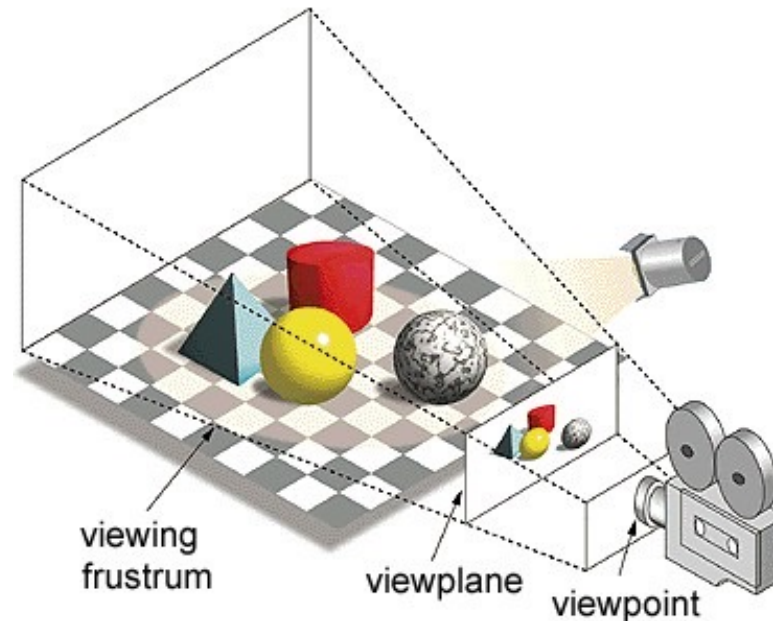
**Gaussian Splats (GS):**  
**Explicit** representation



# Recap: Rendering

In computer graphics, rendering is a function  $g(\theta; c)$  generating an image from a 3D representation  $\theta$  and a specific viewpoint  $c$ .

From Computer Desktop Encyclopedia  
Reprinted with permission.  
© 1998 Intergraph Computer Systems



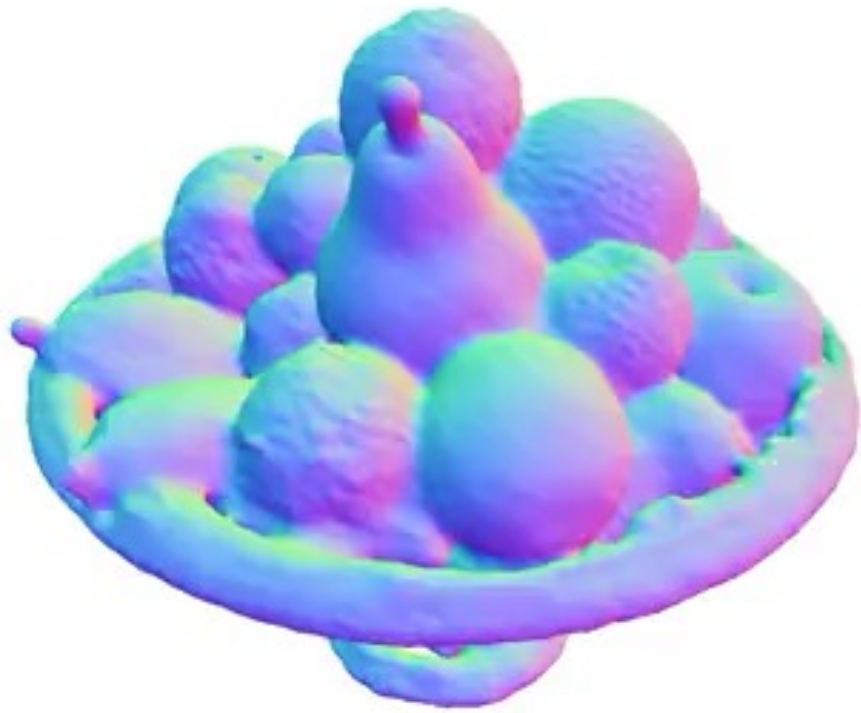
# Timeline

- [2021.12] DreamFields
- [2022.09] DreamFusion
- [2022.11] **Magic3D**
  - **Coarse-to-fine , NeRF -> mesh , diffusion prior 64x64 -> 512x512**
- [2022.12] Latent-NeRF/SJC
- [2023.03] Fantasia3D
- [2023.05] ProlificDreamer
- [2023.08] MVDream/DreamGaussian
- [2024.03] LATTE3D

# Magic3D

Lin et al., "Magic3d: High-Resolution Text-to-3D Content Creation", CVPR 2023.

# Text Prompt -> 3D Mesh Generation



A silver platter piled high with fruits.

# Text Prompt -> 3D Mesh Generation



Michelangelo style statue of an astronaut.

# Prompt-based Editing



A ~~baby bunny~~  
sitting on top of a  
stack of  
pancakes.



A **lego bunny**  
sitting on top of a  
stack of **books**.



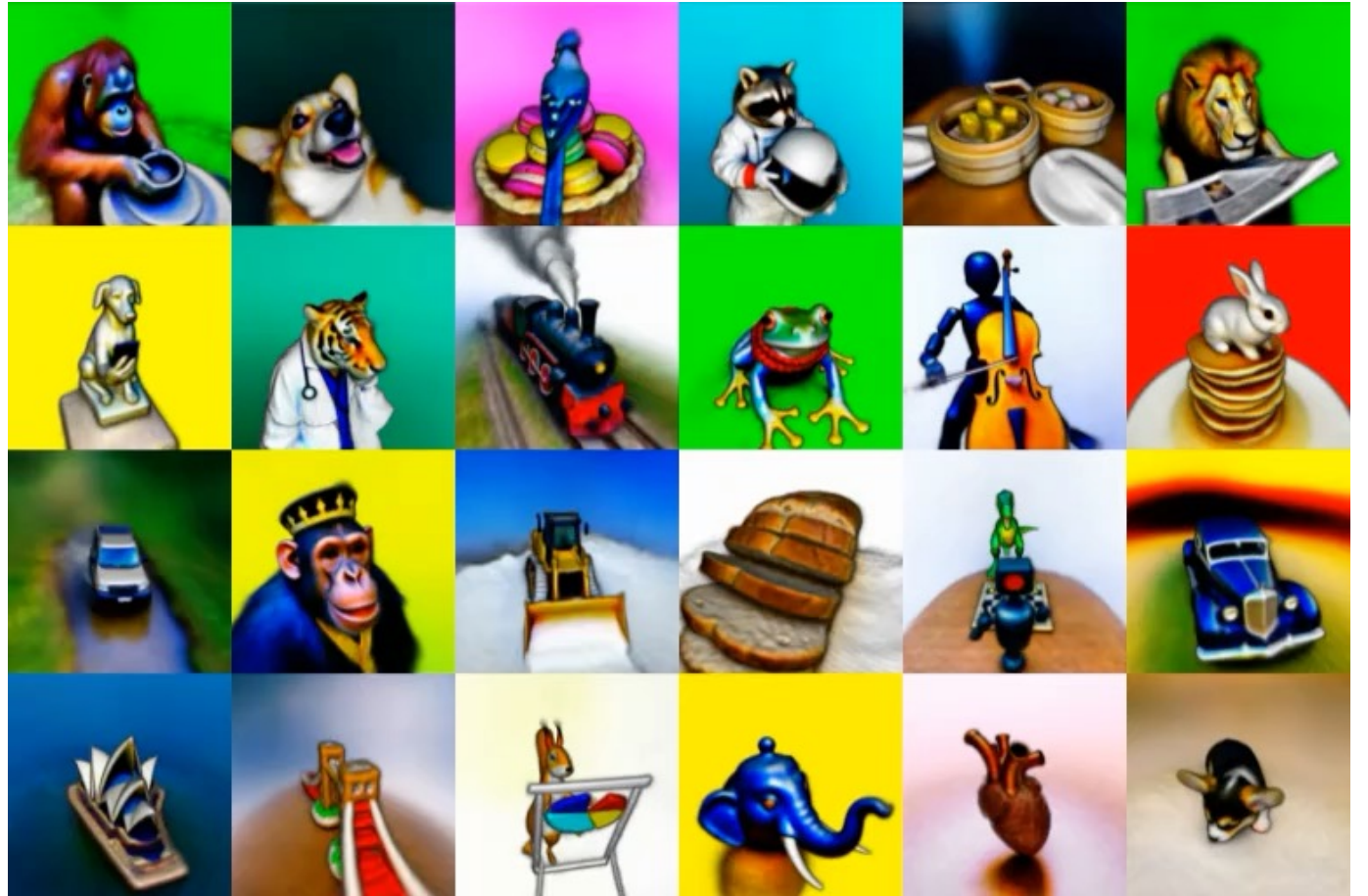
A **metal bunny**  
sitting on top of a  
stack of **broccoli**.



A **metal bunny**  
sitting on top of a  
stack of  
**chocolate  
cookies**.

# DreamFusion 的問題

- 64x64 diffusion prior 太 coarse
- NeRF 的 MLP + volume rendering 導致 optimization 非常耗時
- 一個 prompt 生成一個 3D 物體需要約 1.5 小時



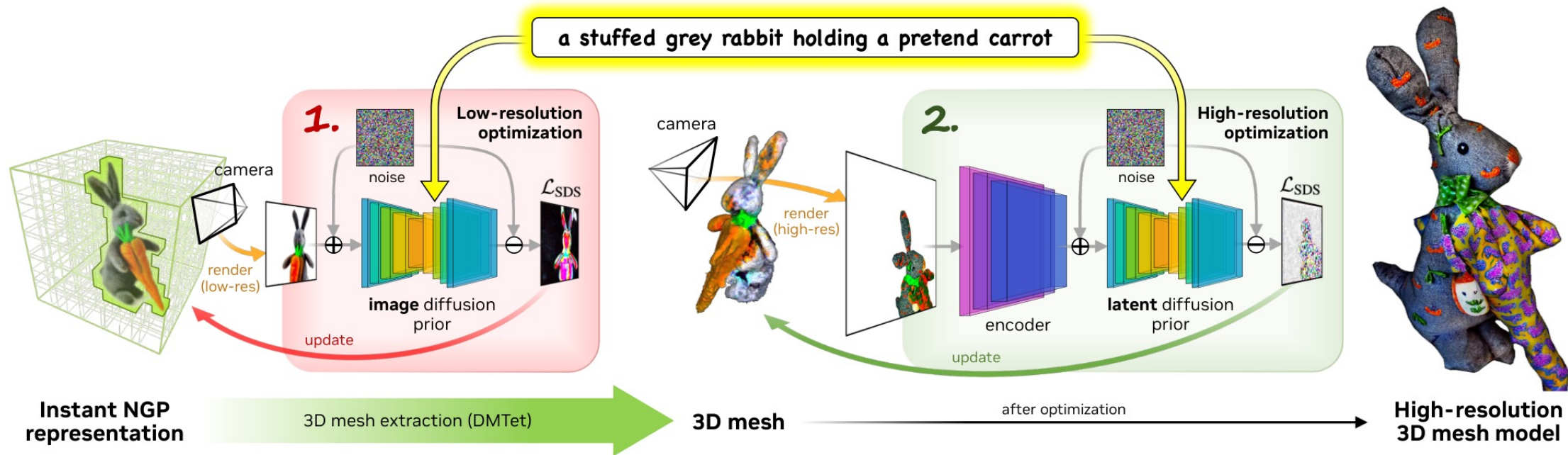
# 3D Representation x Diffusion Prior

|                              | 64x64 image diffusion prior                                                                                                  | 512x512 latent diffusion prior                                                                                                            |
|------------------------------|------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| NeRF<br>volume rendering 速度慢 |  DreamFusion                              |  NeRF 要 render 512x512 的影像會導致 OOM，且 optimization 耗時會更久 |
| Mesh<br>rasterization 速度快    |  Mesh 是由一堆三角型組成的 topology，很難直接被 optimize |                                                                                                                                           |

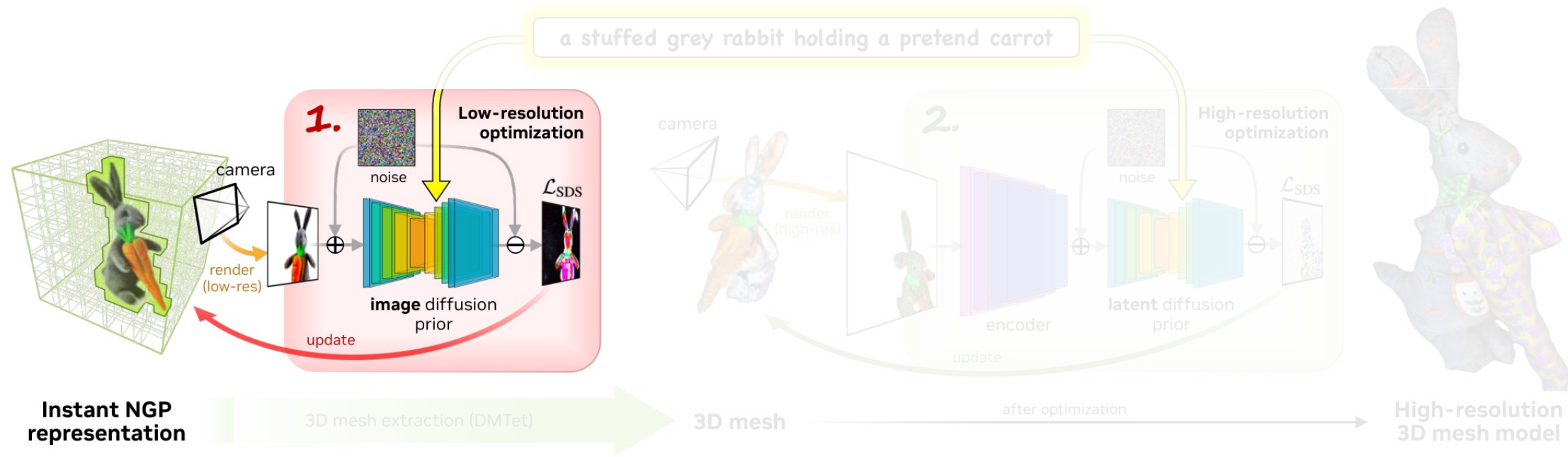
# 3D Representation x Diffusion Prior

|                              | 64x64 image diffusion prior               | 512x512 latent diffusion prior                                             |
|------------------------------|-------------------------------------------|----------------------------------------------------------------------------|
| NeRF<br>volume rendering 速度慢 | ✓ DreamFusion                             | ✗ NeRF 要 render 512x512 的影像會導致 OOM，且 optimization 耗時會更久                    |
| Mesh<br>rasterization 速度快    | ✗ Mesh 是由一堆三角型組成的 topology，很難直接被 optimize | ✓ Magic3D<br>用 NeRF + 64x64 作為初始 -> Mesh 快速 rasterize + 512x512 高解析度 prior |

# Method



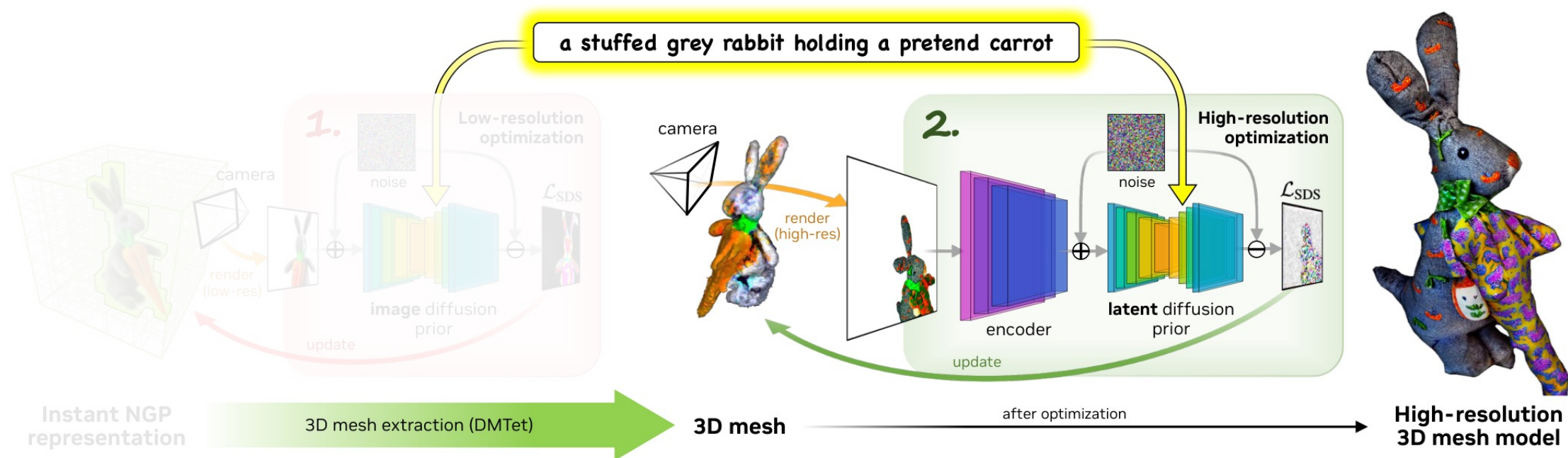
# Method



**Stage 1** ( 約 15 分鐘 ) :

Instant NGP + eDiff-I 的 64x64 prior -> coarse NeRF

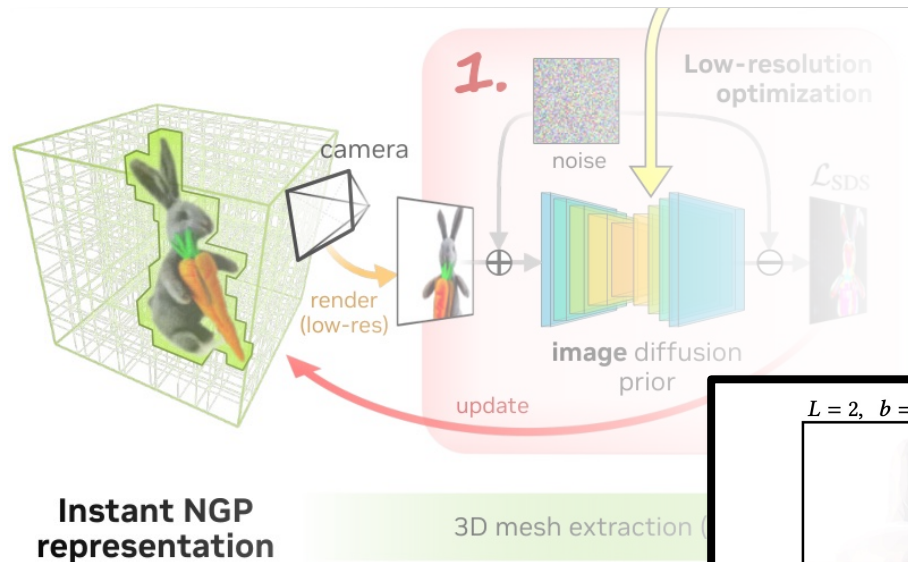
# Method



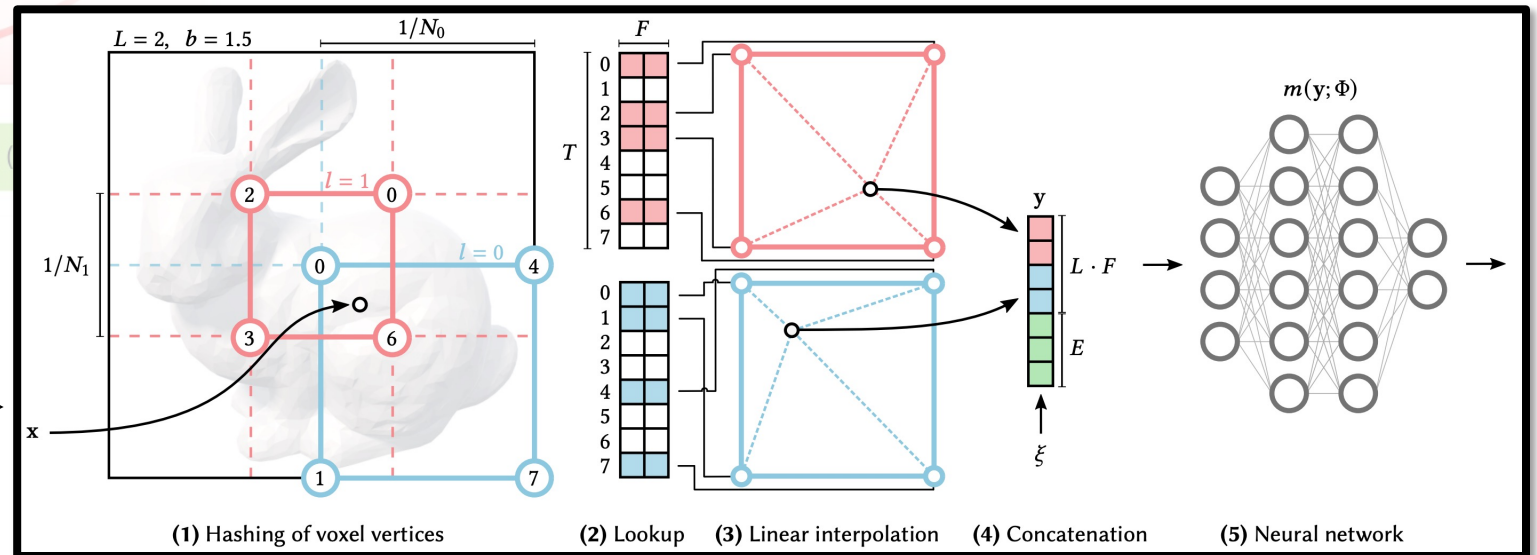
**Stage 2 (約 25 分鐘) :**

DMTet mesh + Stable Diffusion 的  $512 \times 512$  prior -> 高解析度 mesh

# Stage 1: 用 sparse hash Instant NGP 取代 MLP 來加速



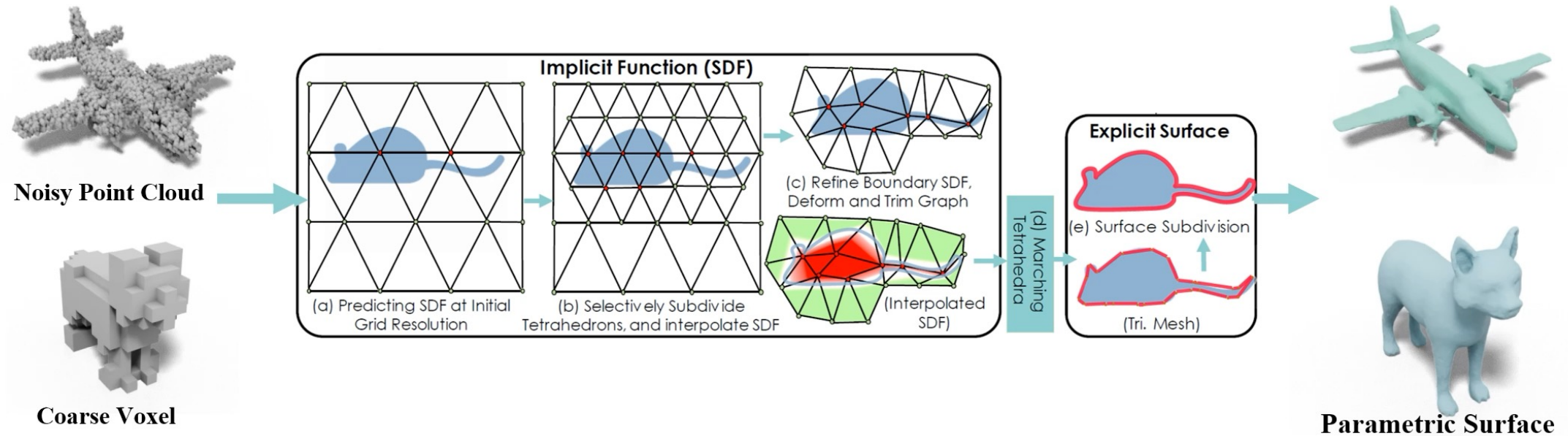
Instant NGP 是一種 explicit representation (NeRF 的 MLP 則是 implicit)，利用 hash 可以快速 query 的特性來大幅加速 optimization



# Stage 2 的 Mesh Representation

## DMTet: NeRF -> Mesh

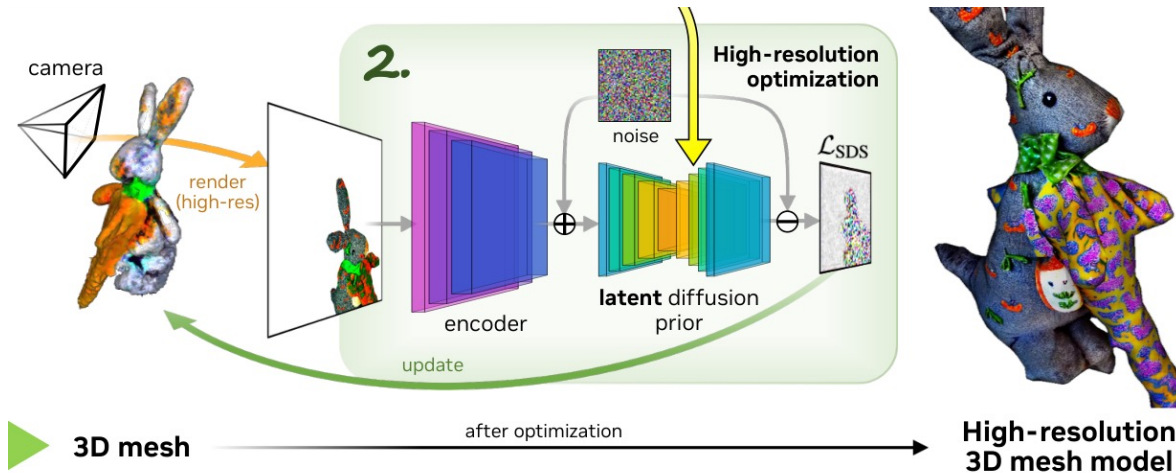
Deep Marching Tetrahedra (DMTet) 是一個可以把 implicit representation (SDF) 轉換成 explicit mesh 的 pre-trained model



# Stage 2 的 Diffusion Prior 在 Latent Space 做 SDS

SDS (pixel space)

$$\nabla_{\theta} \mathcal{L}_{\text{SDS}}(\theta) \\ = (\hat{\epsilon}_{\theta}(\mathbf{x}_t, y, t) - \epsilon_t) \frac{\partial \mathbf{x}_t}{\partial \theta}$$



Magic3D Stage 2 : SDS (latent space)

$$\nabla_{\theta} \mathcal{L}_{\text{SDS}}(\theta) \\ = (\hat{\epsilon}_{\theta}(\mathbf{z}_t, y, t) - \epsilon_t) \frac{\partial \mathbf{z}_t}{\partial \mathbf{x}} \frac{\partial \mathbf{x}}{\partial \theta}$$

$$\mathbf{x} = g(\theta; c)$$

$$\mathbf{z} = \mathcal{E}(\mathbf{x})$$

# Score Distillation Sampling (SDS)

$$\text{Let } \mathbf{x}_t = \sqrt{\bar{\alpha}_t} g(\boldsymbol{\theta}; c) + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\varepsilon}_t$$

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}_{\text{DF}}(\boldsymbol{\theta}) = \frac{\partial}{\partial \boldsymbol{\theta}} \|\hat{\boldsymbol{\varepsilon}}_{\boldsymbol{\theta}}(\mathbf{x}_t, y, t) - \boldsymbol{\varepsilon}_t\|^2$$

$$= 2(\hat{\boldsymbol{\varepsilon}}_{\boldsymbol{\theta}}(\mathbf{x}_t, y, t) - \boldsymbol{\varepsilon}_t) \underbrace{\frac{\partial \hat{\boldsymbol{\varepsilon}}_{\boldsymbol{\theta}}(\mathbf{x}_t, y, t)}{\partial \mathbf{x}_t}}_{\text{Noise Predictor Jacobian}} \frac{\partial \mathbf{x}_t}{\partial \boldsymbol{\theta}} \quad \text{Chain Rule}$$

0

原本沒有忽略 noise predictor Jacobian 計算的叫做 denoising score matching (**DSM**) loss

# Stage 2 的 Noise Sampling

SDS  $\sim$  DDIM reverse process 的一個 step，noise 大會淹沒現有的結構，noise 小可以 refine 細節，因為 stage 1 已經有好的幾何結構了，stage 2 只需要用小的 noise refine 細節就好

**DreamFusion**

$t \sim \mathcal{U}(0.02, 0.98)$

**Magic3D Stage 1**

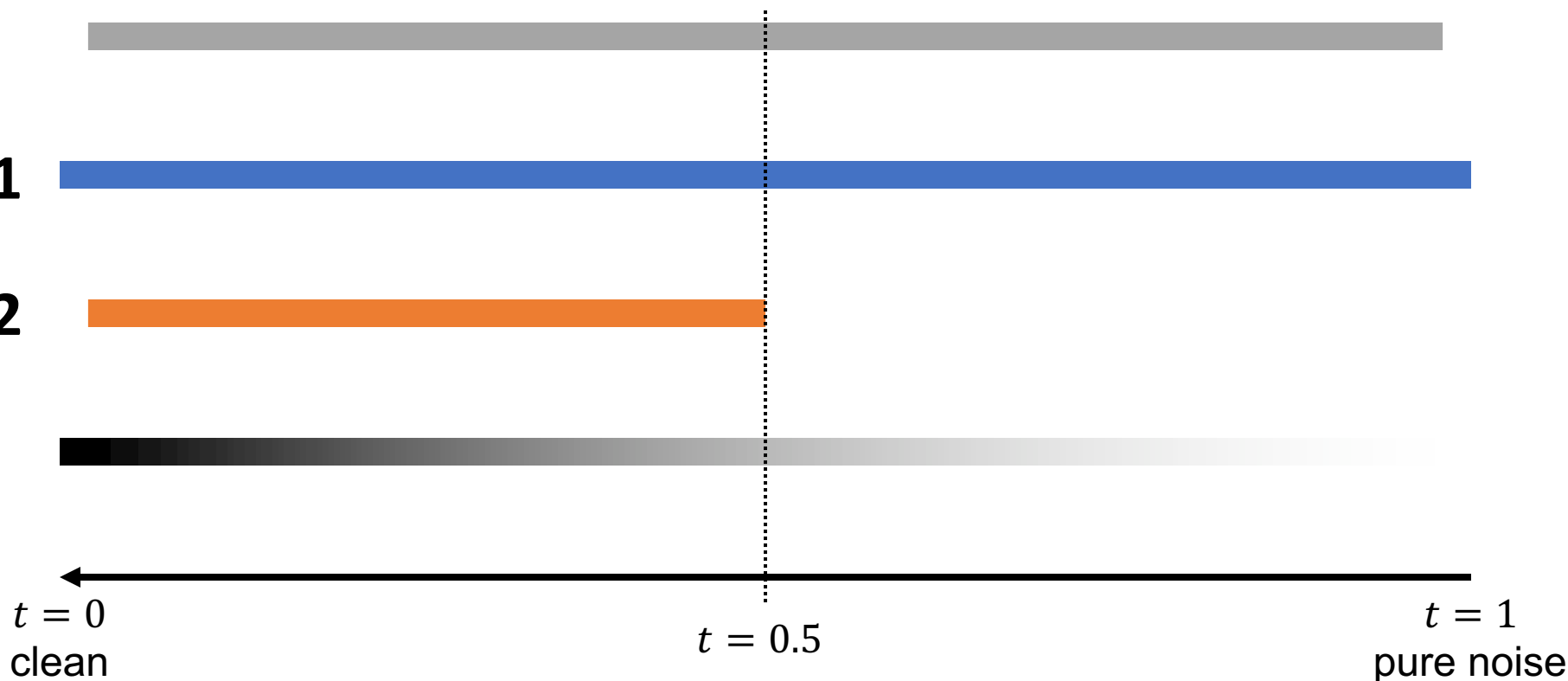
$t \sim \mathcal{U}(0, 1)$

**Magic3D Stage 2**

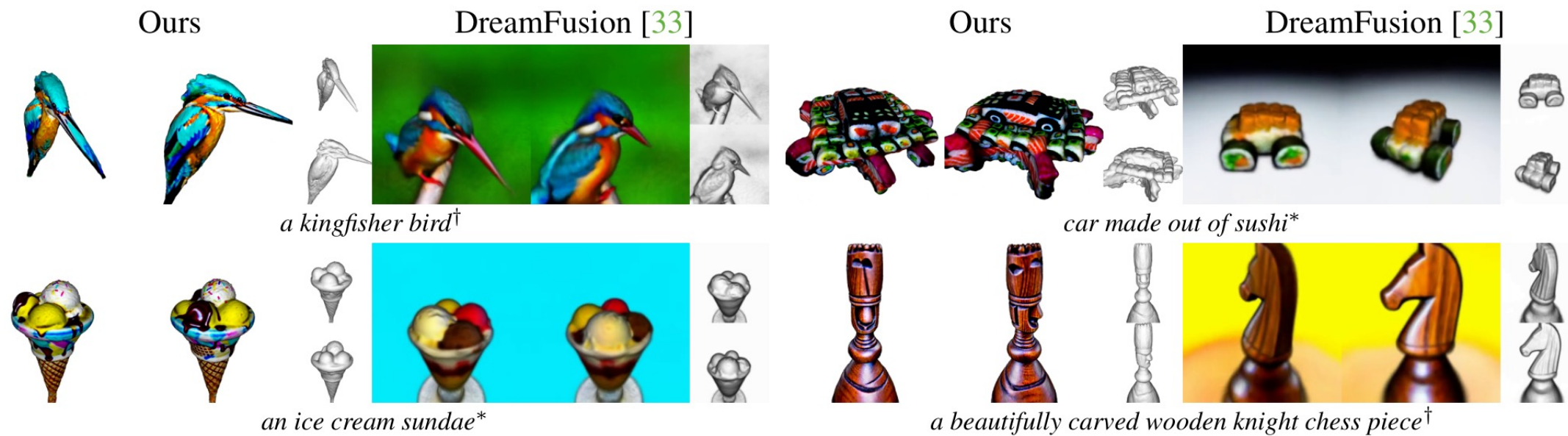
$t \sim \mathcal{U}(0.02, 0.5)$

**SDI**

$t$  遞減



# Experiments



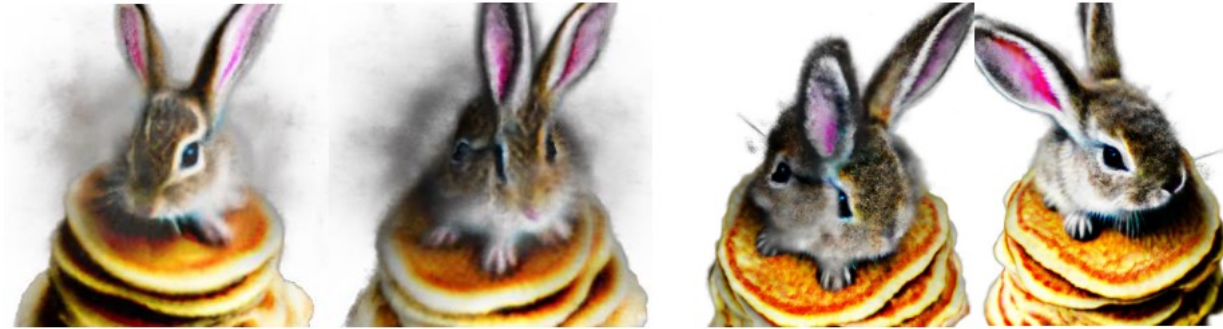
# User Preference Studies

**Table 1. User preference studies.** We conducted user studies to measure preference for 3D models generated using 397 prompts released by DreamFusion. Overall, more raters (61.7%) prefer 3D models generated by Magic3D over DreamFusion. The majority of raters (87.7%) prefer fine models over coarse models in Magic3D, showing the effectiveness of our coarse-to-fine approach.

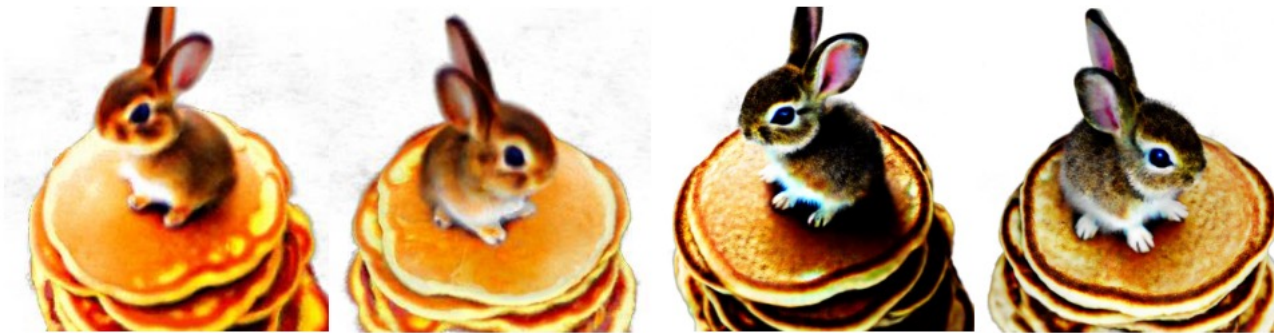
| Comparison                                   | Preference |
|----------------------------------------------|------------|
| Magic3D vs. DreamFusion <a href="#">[33]</a> |            |
| • More realistic                             | 58.3%      |
| • More detailed                              | 66.0%      |
| • More realistic & detailed                  | 61.7%      |
| Magic3D vs. Magic3D (coarse only)            | 87.7%      |

# Ablations

*a baby bunny sitting on top of a stack of pancakes<sup>†</sup>*



(a) Single-stage model



(b) Coarse-to-fine model

# Ablations

**Coarse NeRF**



**Fine-tuned with SR Diffusion Prior**



$t_{max} = 1.0$



$t_{max} = 0.7$



$t_{max} = 0.4$

**Ours**



# Ablations

Stage 2 換成 NeRF 也 OK，但 mesh 的細節還是比較多



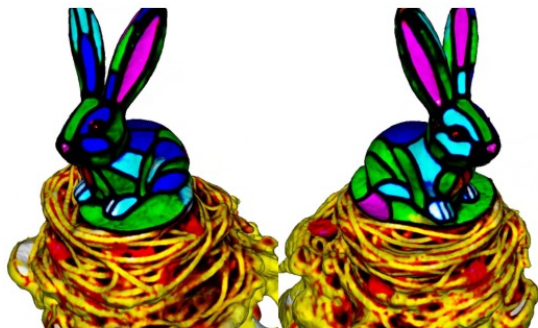
# Prompt-Based Editing

先用 base prompt 產生 coarse NeRF，改 prompt 並且用 latent diffusion fine-tune NeRF，最後再 optimize high-resolution mesh

low-resolution  
before edited



a *baby bunny*  
sitting on top  
of a *stack of  
pancakes*<sup>†</sup>



*stained glass bunny*, a *plate of spaghetti*



*metal bunny*, a *stack of carrots*



*squirrel*, a *stack of books*



a *squirrel*  
wearing a  
*leather jacket*,  
riding a  
*motorcycle*



*bunny*, *broomstick*



*cat*, *rocking horse*



*rat*, *scooter*

# Prompt-Based Editing

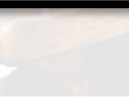
先用 base prompt 產生 coarse NeRF，改 prompt 並且用 latent diffusion **fine-tune NeRF**，最後再 optimize high-resolution mesh

low-resolution  
before edited



**Q.** 為什麼要先 fine-tune NeRF？為什麼不像 stage 2 直接 refine mesh？

leather jacket,  
riding a  
motorcycle



bunny, broomstick

cat, rocking horse

rat, scooter

# Prompt-Based Editing

先用 base prompt 產生 coarse NeRF，改 prompt 並且用 latent diffusion **fine-tune NeRF**，最後再 optimize high-resolution mesh

low-resolution  
before edited

**Q.** 為什麼要先 fine-tune NeRF？為什麼不像 stage 2 直接 refine mesh？

**A.** 因為直接 optimize mesh 只能修改 appearance，topology/geometry 只會微幅變形，所以這裡要從 coarse model (NeRF) 來大幅改變物體的幾何形狀

leather jacket,  
riding a  
motorcycle

bunny, broomstick

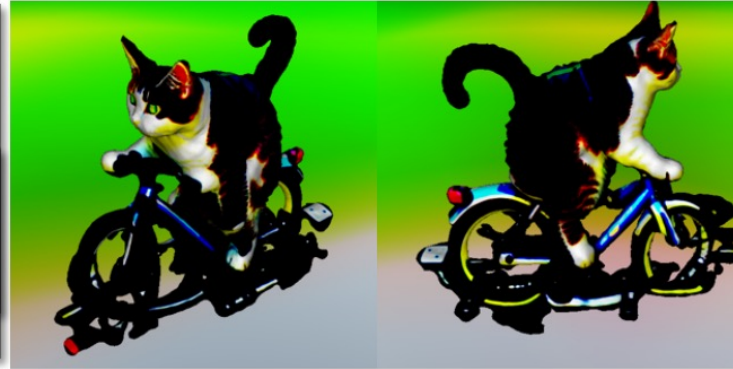
cat, rocking horse

rat, scooter

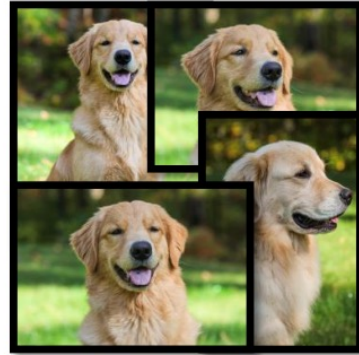
# Experiments: DreamBooth Personalization



Input images



*a [V] cat riding a bike\**

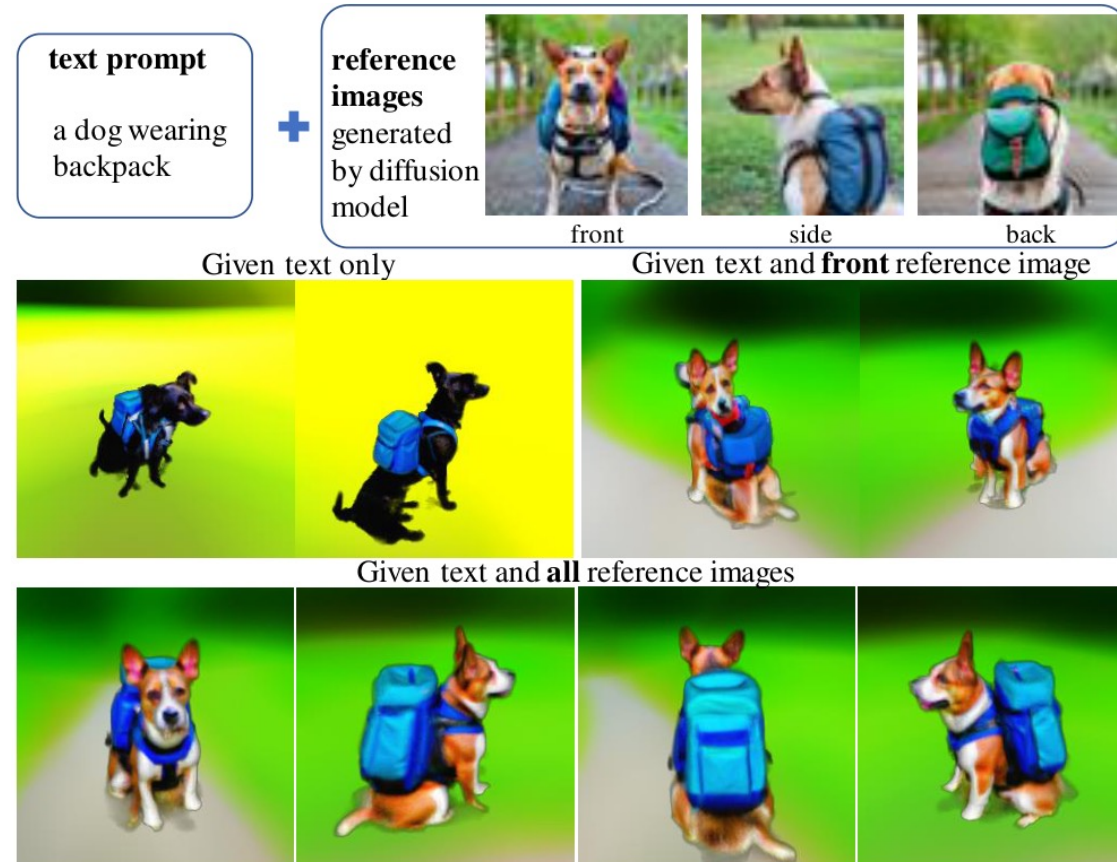


Input images



*a [V] dog running down the track\**

# Experiments: Condition on Reference Images



# Limitations

- Appearance 只有顏色、沒有材質
  - **Fantasia3D**: geometry 與 appearance 分開 optimize，appearance 用 material BRDF 來 model，所以可以做到 relighting
- 仍是 SDS，CFG 調大就會有 oversaturate artifacts，且缺乏多樣性
  - **ProlificDreamer**: VSD loss
- Janus problem
  - **MVDream**: 直接用 3D data train multi-view diffusion
- 每個 prompt 生成物體依然要 40 分鐘 (8 張 A100)
  - **DreamGaussian**: 利用 3DGS 的 real-time rendering 特性來加速 optimization
  - **LATTE3D**: 用一個 generalizable model 在 training 時學到大量的 prompts

# Summary

- [2022.09] DreamFusion: SDS 用 diffusion prior 引導 NeRF
  - 64x64 diffusion prior , optimization 需要 1.5 小時
- [2022.11] **Magic3D** (CVPR 2023): Coarse-to-fine , 512x512 diffusion prior , optimization 加速 2 倍 , output mesh
  - SDS oversaturation/Janus artifacts , optimization 仍然需要 40 分鐘
- [2023.05] ProlificDreamer: VSD 解決 oversaturation 且多樣性與細節更好
  - 要多 train LoRA , optimization 更久
- [2023.08] MVDream: 直接用 multi-view prior 大幅緩解 Janus
  - 要有 3D training data
- [2023.09 -> 2024.03] DreamGaussian -> LATTE3D: 加速
  - Quality 下降 , 生成品質與速度會是一個 trade-off
- [2024] SDI: 用 DDIM inversion 取代 random noise
  - 每個 step 都需要多做一次 DDIM inversion , 速度較慢

# Summary

## What's next?

TRELLIS/TRELLIS.2/SAM 3D 直接在 3D latent 上做 diffusion  
有了大規模 3D data 之後，我們還需要 SDS 嗎？

- [2022.12] SAM 3D
  - 6
- [2023.01] opti
  - S
- [2023.05] Prolif
  - 要多 train LoR
- [2023.08] MVD
  - 要有 3D traini
- [2023.09 -> 2023.10]
  - Quality 下降
- [2024] SDI: 用 D
  - 每個 step 都需



細節更好